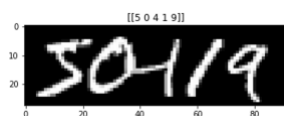


Practical work 4: Embedded Training NN, RNN and CNN with the CTC Loss

In this session we focus on using embedded training thanks to the CTC loss implemented with pytorch. We compare different Neural Network architecture : NN, RNN and CNN.

As for the previous practical work 3 and 4, we focus on images of strings of digits that are of size $28 \times L$ as shown on figure below. $L < 140$ is the image width in pixels and may vary from one image to another because the digits may overlap, which makes the segmentation almost impossible without recognition.

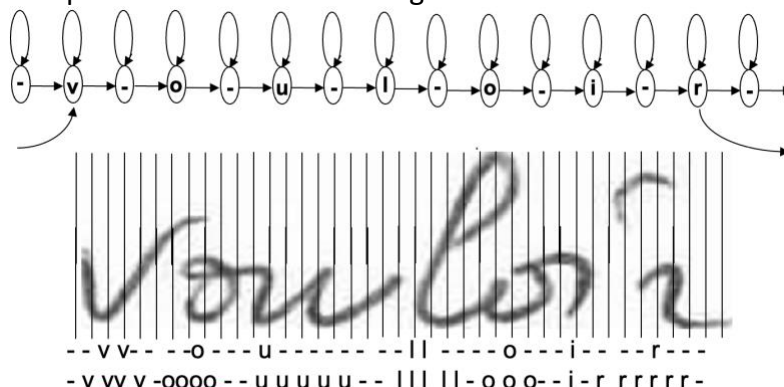
The ground truth is the sequence of classes and does not provide any information about the position of the symbols in the image. Thus, training will proceed without segmentation with embedded training as seen in practical work 3 and 4. We consider the pixel columns of the image as the input frames of the sequences.



Example of a 5 digits sequence generated from the MNIST dataset.

CTC automaton and CTC loss

We recall that the CTC loss uses an alignment process of the frame sequences with the ground truth sequence of symbols, so that a prediction is made about the ground truth frame by frame. This is very similar to using Embedded training with HMM where each symbol is modeled by a single state, introducing a blank state between successive symbols. Contrary to using probabilistic automaton (transition probability matrices) the CTC loss is using binary transitions. Any acceptable transition has a weight of 1.



Thanks to this automaton the CTC loss computes the **cross-entropy loss** between the frame probability distribution inferred using the CTC automaton with the forward backward variables $P(\pi_t = l(j)|X) = \alpha(t, j)\beta(t, j)$, and the current probability distribution of the output of the Neural network at frame position t . Where the forward and backward variables are defined as follows:

$$y_{l(j),t} = P(l(j)|x_1 \dots x_t)$$

$$\alpha(t, j) = P(l(1:j)|x_1 \dots x_t)$$

$$\alpha(t, j) = \left(\sum_{i=1, \dots, 2U+1} \alpha(t-1, i) a_{ij} \right) y_{l(j),t}$$

$$\beta(t, i) = P(l(i:2U+1)|x_{t+1} \dots x_T)$$

$$\beta(t, i) = \sum_{j=1, \dots, 2U+1} \beta(t+1, j) a_{ij} y_{l(j),t+1}$$

- **As a consequence** the CTC loss modules of pytorch requires augmenting the ground truth sequences of symbols by inserting the blank symbol between every successive symbols
- **As a consequence** during inference, the neural network will provide the probability distribution of every states for every time step (frame). We need to decode this lattice using the Viterbi best path decoding algorithm. As there is only one state par symbol, **Viterbi decoding reduces to** finding the state with the highest probability at each time step.
- **As a consequence** the best path requires further **post processing** to remove the repetition of the same state, and finally remove every blank label.

Preparing data with pytorch

Comme précédemment, les séquences de 5 digits d'apprentissage et de teste sont chargées en mémoire à partir du fichier *MNIST_5digitsDifficile.pkl*.

Pour utiliser pleinement les fonctionnalités de pytorch il est préférable d'adopter le mode de gestion par lots intégré dans pytorch. La classe **DataLoader** est faite pour cela. Elle permet notamment de préparer les lots et de les mettre au bon format (**Tensor**) pour les mettre à disposition de la boucle de calcul. Ainsi La préparation des données peut être gérée en parallèle des calculs. Ce ne sera pas exactement le cas ici car toutes les données sont chargées en mémoire initialement.

La **création du dataloader** est réalisée par les lignes suivantes. On procède de même pour la les données de valid et de test.

```
train = x_train[:N_train_seq]
gt_train = y_train[:N_train_seq]
Train_seq_dataset = DigitSequenceDataset(train, gt_train)
train_dataloader = torch.utils.data.DataLoader(Train_seq_dataset,
                                                batch_size = config['batch_size'],
                                                shuffle=True,
                                                collate_fn = pad_collate)
```

Question 1 : Que réalise la fonction `pad_collate` ?

Question 2 : Que réalisent les lignes de code suivantes ? Expliquer l'organisation des données dans un lot.

```
for batch in (train_dataloader):
    for i in range(5):
        plt.imshow(np.flip(batch[0][:,i,:].numpy().T,axis=0), cmap='gray')
        plt.title(batch[1][i])
        plt.show()

    break
```

Architecture du réseau de neurones

Les déclarations de la classe et des méthodes nécessaires à l'instanciation d'un réseau de type LSTM ou BLSTM sont faites dans le fichier **CTC.py**.

Question 3 : Quelle est l'architecture du réseau par défaut ? Quels sont les paramètres à modifier pour passer d'un réseau LSTM à un réseau BLSTM ?

Question 4 : Quels sont les paramètres à modifier pour créer un LSTM à deux couches ?

Question 5 : Comment modifier ce code pour utiliser un MLP à une couche comme dans le TP précédent ?

Entraînement du réseau de neurones

Question 6 : dans la boucle d'apprentissage, expliquer les lignes de code suivantes. Pourquoi la log softmax doit-elle être utilisée ?

```
pred = torch.nn.functional.log_softmax(model(X.float()),dim = 2)
loss = loss_fn(pred, y,X_l,y_l)
```

Question 7 : Quelle est l'optimiseur utilisé par défaut ?

Question 8 : Expliquer les étapes de la boucle principale d'apprentissage dans le code **Main-CTC.py**.

Reconnaissance de séquences

Comme indiqué plus haut, l'étape de reconnaissance consiste à propager les données dans le réseaux de neurones pour sélectionner la sortie la plus probable à chaque pas de temps qui appartient au meilleur chemin accepté par l'automate CTC.

Question 9 : Compléter la fonction `statesToSymbols()` ci-dessous pour qu'elle réalise l'extraction de la séquence de symboles reconnus dans la meilleure séquence d'états prédite.

```
def StatesToSymbols(best_path,T,blank):  
    .....  
    return bs
```

Expérimentations

Réaliser un ensemble d'expériences avec différentes architectures mettant en évidence les forces et faiblesses de ces architectures au regard des performances en reconnaissance lorsque les réseaux sont entraînés sur le dataset limité à 2000 séquences ($N_{\text{train}} = 10\,000$).

Question 10 : Observer l'allure typique de l'évolution de la loss CTC avec l'optimiseur ADAM, expliquer le phénomène .

Question 11 : Compléter le tableau ci-dessous

Archi	N cellules	Batch_size	Lr =	Test CER (%)
LSTM, 1 couche	128	32	1e-3	
	256	32	1e-3	
LSTM, 2 couches	128	64	1e-3	
	256			
BLSTM, 1 couche	128	64	1e-3	
BLSTM, 2 couches	128	64	1e-3	
	256	64	1e-3	

Question 12 : Mettre en œuvre une architecture uniquement convolutive, puis réaliser des expériences mettant en évidence ses performances en les comparant aux performances de l'architecture LSTM et BLSTM.

Question 13 : Reprendre la question précédente en mettant en œuvre une architecture hybride CNN-BLSTM. Analyser les performances et conclure.